SPORTS MANAGER

Milestone 1: Design and Architecture Document

CE-216 Computer Engineering Project

Instructor

Assoc. Prof. Dr. Kaya Oğuz

Members:

Deniz Karaman - 20230602043

Selim Palanduz - 20230602056

Yusuf Emir Yılmaz - 20230602078

Mustafa Şen - 20230602062

MARCH 2026

Introduction

1.1- Sports management games have been popular among players for many years. These games allow users to take the role of a manager who is responsible for organizing a team, selecting players, deciding tactics, and competing against other teams in a league. One of the most well-known examples of this genre is Football Manager, which allows players to manage football clubs and simulate entire seasons. Earlier versions of such games relied heavily on text-based match simulations and statistical data rather than graphical match engines. Despite their simplicity, these games remained engaging due to the strategic decisions related to team selection, player development, and tactical planning.

1.2- Inspired by this concept, the aim of this project is to develop a simplified sports manager application with a graphical user interface (GUI). The application will allow the user to manage a team within a league system, view player information, arrange lineups, and simulate matches throughout a season. Unlike traditional manager games that focus on a single sport, this application is designed to support multiple sports. The system will initially be implemented using football as the primary example, but its architecture will allow additional sports to be integrated later without major modifications.

1.3 -To demonstrate the flexibility of the system, the project description introduces a fictional sport called “Headball.” In this example, teams compete in a league where each team consists of players with sport-specific positions and attributes. Matches are simulated over multiple quarters, and managers can adjust tactics or make substitutions between periods. League standings are calculated based on match results, and tie-breaking rules are applied when teams have the same number of points. Although the example uses a fictional sport, the same structure can easily be adapted for real sports such as football, basketball, or volleyball.

1.4 -A key requirement of this project is the use of object-oriented design principles to ensure extensibility and maintainability. The system will be designed around abstractions so that the core application does not depend on the specific rules of any particular sport. Instead, sport-related behaviors will be implemented through interfaces and concrete classes. This design approach allows new sports to be added to the system with minimal changes to the existing codebase.

1.5- In addition, the application will include a clean and modern user interface that allows users to interact with the system easily. Teams, players, and coaches will be generated dynamically, and visual elements such as team logos and player names can be created using AI-generated data. By combining a modular architecture with a simple yet functional user interface, the project aims to demonstrate the effective use of object-oriented programming concepts in the development of a flexible sports management simulation.

System Overview

2.1 The purpose of this section is to explain the overall structure and workflow of the sports manager game. It describes how the game starts, how leagues and teams are generated, and how the user interacts with the system during a season.

2.2- The general flow of the game is as follows:

1-The user starts a new game.
2-The user selects the sport to manage.
3-The system generates a league with several teams.
4-Each team is created with randomly generated players and coaches.
5-A season schedule is generated.
6-During the week, teams perform training sessions.
7-During the weekend, matches are played.
8-The league standings are updated after each match.
9-At the end of the season, the league winner is determined.

Architecture Design

The application is designed using a layered architecture in order to keep the user interface separate from the game logic and sport specific rules. This structure improves maintainability, readability, and extensibility of the system. By separating the responsibilities into different layers, the system can support multiple sports while keeping the user interface consistent.

3.1-User Interface Layer
The User Interface (UI) layer is responsible for all interactions with the user. It displays information such as league tables, team rosters, match schedules, and match results. The UI allows the player to select tactics, manage players, and navigate through the game.
This layer does not contain any sport specific logic. Instead, it communicates with the application layer to retrieve and display data.

3.2-Application Layer
The Application layer controls the overall flow of the game. It manages the progression of the season, coordinates matches, and handles interactions between the user interface and the sport logic.
This layer acts as a controller that connects the UI with the core game functionality.

3.3-Sport Logic Layer
The Sport Logic layer contains the rules and mechanics specific to each sport. Since the application must support multiple sports, this layer is designed using abstraction. A common interface called Sport defines the basic functionality required by all sports.
Each sport implements this interface and provides its own rules, match simulation logic, and player attributes.

Examples:
- Sport (interface)
- FootballSport
- VolleyballSport (future)

3.4-Data Layer
The Data layer contains the core entities of the game such as players, teams, leagues, and matches. These classes represent the main objects used by the system and store the data required for the game simulation.

Class Examples:
- Player
- Team
- Coach
- Match
- League

Layer Interaction

The interaction between layers follows a clear structure. The user interacts with the User Interface layer, which sends requests to the Application layer. The Application layer then calls the appropriate methods in the Sport Logic layer. The Sport Logic layer uses the Data layer to access and update the game entities.

Abstraction and Interfaces

Why We Need Abstraction

In this project, abstraction is used to make the system flexible and extensible. Since the application is designed to support multiple sports, the core system should not depend on the specific rules of a single sport. Instead, the program defines common behaviors through interfaces and abstract structures. This allows new sports to be added to the system without modifying the existing architecture.

Sport Interface

To achieve this flexibility, a common interface called Sport is defined. This interface represents the general behavior that every sport in the system must implement. The user interface and application layer interact with this interface rather than a specific sport implementation.

4.1-Sport Interface Responsibilities

Method	**Description**
getSportName()	Returns the name of the sport (e.g., "Headball").
getNumberOfPeriods()	Defines how many periods a match has (e.g., 2 halves for football, 4 quarters for headball).
getPointsForWin()	Returns the number of points awarded to a team for a win.
getPointsForDraw()	Returns the number of points awarded for a draw.
getAllowedSubstitutions()	Returns the maximum number of player substitutions allowed in a single match.
getSquadSize()	Defines the number of players allowed on the field/court.

These methods represent the core behaviors that every sport must implement. Different sports may implement these operations differently depending on their rules and match mechanics.

4.2-IMatchEngine Interface

IMatchEngine abstracts the match simulation. It processes one period at a time, allowing the UI to present results after each quarter or half before the player makes tactical changes. Injury checking is also delegated to the engine.

setupMatch(Match match)	Initialises the engine with the two teams and their lineups/tactics.
simulateNextPeriod()	Simulates a single period and returns the score increment.
isMatchOver()	Returns true when all periods have been**.**
checkInjuries()	Returns players injured during the last simulated period.

Concrete Sport Implementations

Concrete sport classes implement the Sport interface and define their own rules, player attributes, and match mechanics. For example, a football implementation may include specific tactics, formations, and scoring rules. In the future, other sports such as volleyball or basketball can be added by implementing the same interface.

This design ensures that new sports can be added to the system without modifying the existing user interface or application logic.

Main Classes

This section describes the main classes used in the system. These classes represent the core entities of the sports manager game and are responsible for managing teams, players, matches, and the league structure.

5.1-Player

The Player class represents an athlete that belongs to a team. Each player has attributes that define their abilities and current status in the game.

Main responsibilities

- Storing player information
- Managing player attributes
- Tracking injuries and availability
-

Possible attributes

- name
- position
- skill attributes
- injury status

5.2-Team

The Team class represents a sports team that competes in the league. Each team contains a group of players and may have its own tactics and strategy.

Main responsibilities

- Managing players in the team
- Applying tactics and strategies
- Participating in matches
-

Possible attributes

- team name
- player list
- tactics

5.3-Coach

The Coach class represents the coaching staff responsible for improving player performance through training.

Main responsibilities

- Managing training sessions
- Improving player attributes

Possible attributes

- coach name
- training skill

5.4-Match

The Match class represents a game played between two teams. It handles the simulation of the match and stores the results.

Main responsibilities

- Simulating the match
- Tracking scores
- Recording match events

Possible attributes

- home team
- away team
- score
- match events

5.5-League

The League class represents the competition structure where teams compete during a season.

Main responsibilities

- Managing teams in the league
- Scheduling matches
- Tracking league standings

Possible attributes

- team list
- match schedule
- league table

5.6-GameManager

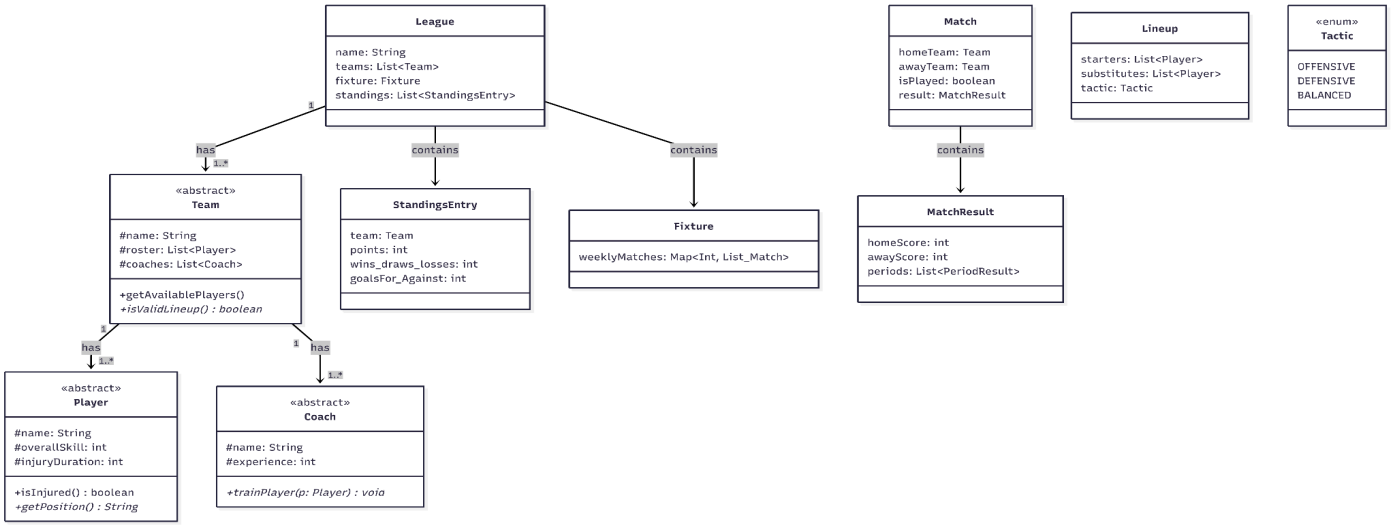
The GameManager class controls the overall flow of the game. It manages the progression of the season and coordinates interactions between different parts of the system.

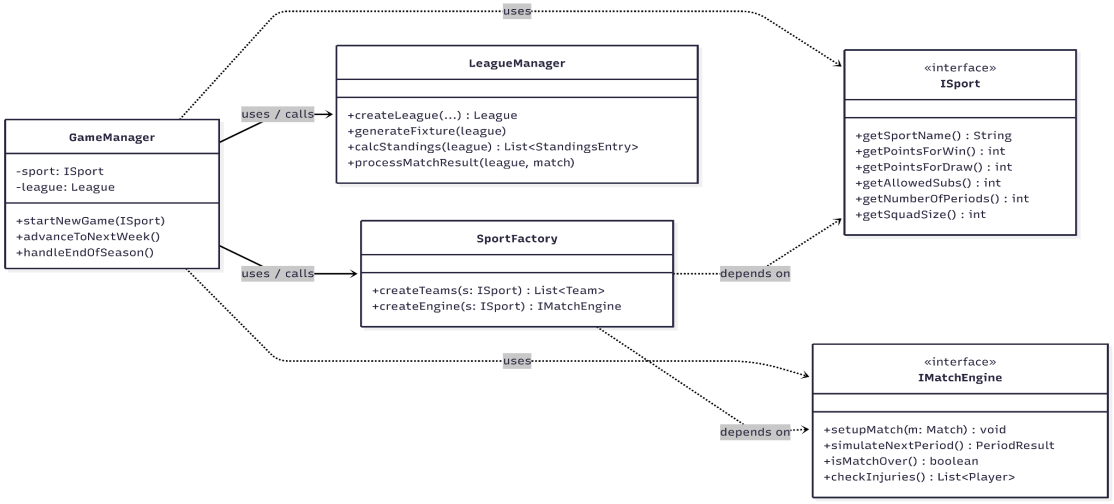
Main responsibilities

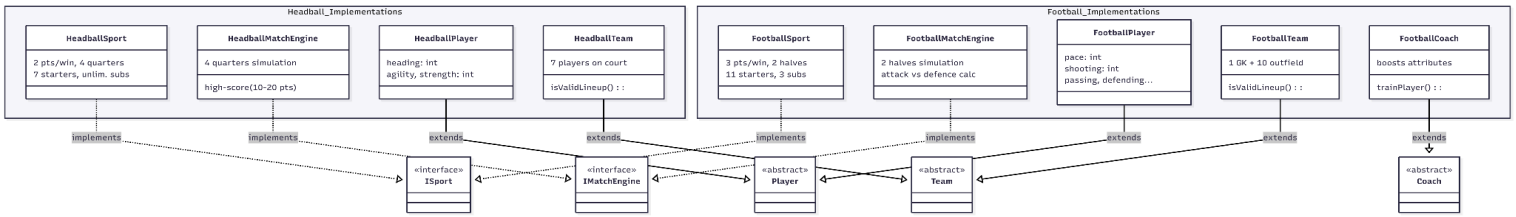
- Managing the game flow
- Controlling seasons and matches
- Connecting the UI with the core game logic

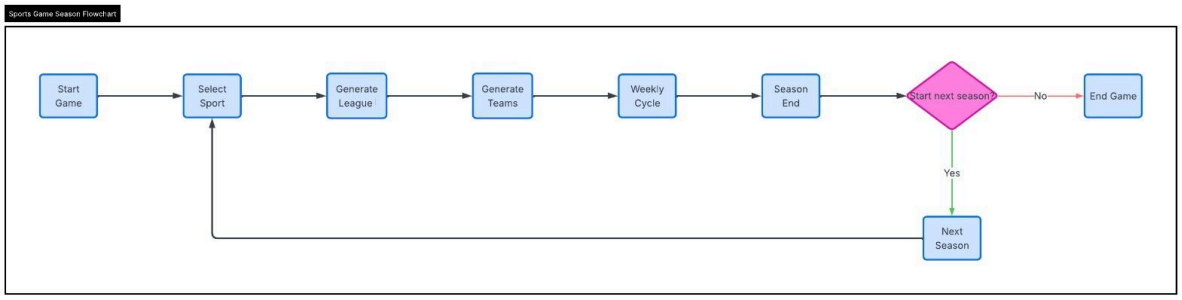
UML Class Diagram

6.1-The UML class diagram illustrates the main classes of the system and their relationships. It provides a visual representation of the structure of the application. The diagram shows how the core entities such as players, teams, matches, and leagues are connected. It also demonstrates the use of abstraction through the Sport interface, which allows the system to support different sports implementations.









User Interface Design

The user interface of the application is designed to be simple, clean, and easy to use. The interface allows the player to navigate through different parts of the game such as team management, league standings, and match simulations. The UI is designed separately from the game logic so that the core functionality of the system remains independent from the visual components.

7.1-Main Menu

The main menu is the starting point of the application. From this screen, the user can start a new game or exit the application. When a new game is selected, the system asks the user to choose the sport to manage.

7.2-Team Management Screen

The team management screen allows the player to view the team roster and manage players. The user can review player attributes, adjust tactics, and prepare the team for upcoming matches.

7.3-League Table Screen

The league table screen displays the standings of all teams participating in the league. The table shows information such as points, matches played, wins, losses, and score differences.

7.4-Match Screen

The match screen shows the progress of a match between two teams. The system simulates the match and displays the score and key events. After each quarter, the player can make tactical adjustments or substitute players.

The user interface communicates with the application layer but does not contain sport-specific logic. This separation ensures that the UI can remain the same even if new sports are added to the system.

Extensibility

- The system is designed to be extensible so that new sports can be added in the future without modifying the existing architecture. This is achieved by using abstraction and interfaces in the design of the application. The system interacts with sports through a general interface rather than relying on a specific sport implementation.
- By defining a common Sport interface, different sports can implement their own rules, player attributes, and match mechanics while still following the same overall structure. For example, while football may have different tactics and scoring systems compared to volleyball or basketball, all sports will follow the same core workflow of league creation, team management, and match simulation.
- To add a new sport, developers only need to create a new class that implements the Sport interface and defines the rules and behavior specific to that sport. This approach ensures that the user interface and the main application logic do not need to be changed when new sports are introduced.
- This design improves the maintainability and scalability of the system while following object-oriented programming principles.

Data Generation

- The game requires various types of data such as player names, team names, and team logos. Instead of manually creating all these elements, the system can generate them automatically when a new game starts.
- Player and coach names can be generated using predefined datasets or AI-based name generators. This allows the game to create a large number of unique characters without requiring manual input. Similarly, team names can be selected from a predefined list or generated dynamically.
- In addition, team logos can be loaded from a collection of images and assigned randomly to teams to improve the visual appearance of the game. These elements help create a more immersive and varied experience for the player.
- This approach allows the system to quickly generate a complete league with teams, players, and visual assets whenever a new game is created.

Conclusion

- This report presented the design and architecture of the sports manager game. The system is designed using object-oriented principles such as abstraction and modularity in order to support multiple sports within the same application structure. The use of interfaces allows the application to remain flexible and extensible, making it possible to add new sports without modifying the existing user interface or core logic.
- The architecture separates the user interface, application logic, and sport-specific rules, which improves maintainability and scalability. The proposed design provides a strong foundation for the implementation phase of the project in the following milestones.